Information Sciences 649 (2023) 119656

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Toward explicit control between exploration and exploitation in evolutionary algorithms: A case study of differential evolution

Zonghui Cai [a], Xiao Yang [a], MengChu Zhou [b], Zhi-Hui Zhan [c], Shangce Gao [a,*]

[a] Faculty of Engineering, University of Toyama, Toyama-shi, 930-8555, Japan
[b] Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102, USA
[c] School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China

ARTICLE INFO

Keywords:
Differential evolution
Exploration
Exploitation
Explicit control
Numerical optimization

ABSTRACT

Exploration and exploitation are two cornerstones of evolutionary algorithms. An appropriate balance between exploration and exploitation can drive a search process toward global optima with a fast convergence rate. However, this balance is not comprehensively understood, and the issue of how to effectively control it is very challenging. In this paper, a new search framework based on an explicit control strategy that balances the amounts of exploration and exploitation in a search process is proposed. First, an explicit control strategy consisting of three types of transference is proposed to balance exploration and exploitation. Then, exploration and exploitation operators are formally defined by adaptive Gaussian local search with reinitialization and multioffspring-based differential evolution, respectively. Finally, a new triple-transference-based differential evolution method is proposed. The experimental results on 29 benchmark optimization functions show its outstanding performance, especially on complex problems. The balance between exploration and exploitation in the proposed algorithm is discussed in detail. The success of this new framework provides more insights into the principles of balancing exploration and exploitation. It also leads us to believe that exploration and exploitation in evolutionary algorithms can eventually be explicitly controlled.

1. Introduction

Evolutionary algorithms (EAs) are one of the most well-known stochastic generate-and-test search approaches due to their superior performance in continuous and multimodal optimization [1]. Since the inception of EAs, the terms exploration and exploitation have played an essential role in describing the search dynamics of an algorithm. The common understanding is that exploration visits entirely new regions of a search space, while exploitation utilizes the information from previously visited regions to determine promising regions to be searched in a fine-grained manner [2]. Over-emphasizing exploration slows down the convergence rate and cannot ensure the quality of solutions. In contrast, overly stressing exploitation usually causes a fast loss of population diversity and a premature convergence to a local optimum. Thus, a good balance between exploration and exploitation is desired for a good-performing EA [3]. However, the amount of exploration or exploitation needed to find the global optimal solution is problem-dependent. For example, less exploration is certainly preferred in unimodal optimization problems compared with multimodal ones. Since, generally, neither the landscape nor the optimal solution of an optimization problem is known in advance, an appropriate

* Corresponding author.
E-mail address: gaosc@eng.u-toyama.ac.jp (S. Gao).

https://doi.org/10.1016/j.ins.2023.119656
Received 28 April 2022; Received in revised form 29 August 2023; Accepted 1 September 2023

Available online 9 September 2023
0020-0255/© 2023 Elsevier Inc. All rights reserved.

balance between exploration and exploitation should be maintained or controlled during the search process to obtain the optimal solution while achieving faster convergence [4].

In the literature, the control of exploration and exploitation in EAs has been achieved indirectly through parameters and population diversity. Since some parameters may significantly affect the search behavior of EAs, several attempts that properly balance exploration and exploitation by systematically controlling parameters have been tried. Some early approaches start with exploration and then gradually transform into exploitation by changing the mutation rate or population size along with the evolution [5]. However, these time-varying approaches have limitations when facing multimodal problems or dynamic environments. Therefore, several adaptive approaches that utilize information available from previous searches or current population states, such as fitness values, diversity, and entropy, have been proposed [6]. Population diversity has already been recognized as another important factor affecting the performance of EAs. Population diversity refers to differences among individuals, and many measurement methods according to genotype and phenotype levels have been developed [7]. From the perspective of exploration and exploitation, an increase in population diversity indicates that an algorithm is in the former phase, while a decrease in diversity implies that it is in the latter phase. Therefore, exploration operators are usually defined as those that can maintain the diversity of the population. To avoid situations in which a population that is too diverse may result in ineffective exploitation and very slow convergence, several approaches regarding diversity maintenance, diversity control, and diversity learning have been introduced [8].

Population topology has also been studied to balance exploration and exploitation in EAs. In recent research, subpopulations were widely used to maintain the diversity of a population for good exploration. In [9], two separate populations for exploration and exploitation were used to obtain excellent performance in terms of divergence and convergence. In addition to bipopulations, some island-based models have been developed to maintain diversity in multimodal optimization [10]. In these schemes, population diversity is maintained by restricting interactions among individuals. Although they have significant effects on diversity, they are considered only a method of promoting exploration. To improve both exploration and exploitation, some complex structures have been devised. In [11], a three-layered hierarchical structure was adopted to reduce the attraction of the global best individual to the whole population in a gravitational search algorithm. The global best individual in the top layer leads the evolution of the K best individuals in the intermediate layer, and the K best individuals direct the movement of other individuals in the bottom layer. Since different degrees of exploration and exploitation are needed in the different stages of solving a specific problem, some adaptive topology schemes have been devised [12]. While the balance between exploration and exploitation is promoted in these algorithms, we see no explicit control over either phase because the delimitation of exploration from exploitation in the population cannot be clearly observed.

Schemes based on alternating between exploration and exploitation include hybrid EAs and memetic algorithms. In the former, under the guidance of the no-free-lunch theorem [13], an algorithm with powerful exploitation ability is usually hybridized with an algorithm that has the advantage of exploration to achieve both abilities. In [14], a modified differential evolution method is integrated into a modified artificial bee colony algorithm to accelerate the convergence rate, and a catastrophe operator is proposed to balance exploration and exploitation. In [15], genetic operators are embedded in the framework of particle swarm optimization under a cascade architecture. The particles' movements are guided by their exemplars, and the exemplars are updated by crossover and mutation to improve diversity. In memetic algorithms, one or several local operators are usually applied to individuals to further refine a solution by exploiting its neighborhood. In [16], a chaotic local search method is implemented for the best individual obtained by differential evolution to improve the solution quality. To achieve a good trade-off between exploration and exploitation, some problems regarding how often and which individuals should be exposed to local search have been further studied. For example, the probabilistic model considers exploration and exploitation as independent processes from probabilistic perspectives and a theoretical upper bound for individual learning intensity is introduced [17].

Nevertheless, the approaches mentioned above are all implicit and indirect, which means that it is difficult for them to clearly identify exploration and exploitation. The explicit control of exploration and exploitation has already received a great deal of attention. The first explicit metric that measured both exploration and exploitation is proposed in [18]. It utilizes an ancestry tree-based data structure to represent the evolution of the whole population. The exploration tree and exploitation one are split from the ancestral tree through a predefined threshold value, which defines the neighborhood boundary between individuals. Subsequently, this metric is used to adaptively control the parameters of differential evolution for fitting Sovova's model [19]. Instead of using an explicit metric, some direct control approaches such as trigger-based and subpopulation-based methods have been developed to balance exploration and exploitation in EAs. In trigger-based methods, the population alternately performs exploration and exploitation [20]. Usually, the exploration phase of EAs is conducted first to discover potential search areas; thereafter, the exploitation phase is fine-tuned to find better solutions. Unlike this serial model, both phases are performed in parallel in subpopulation-based methods. In particular, in a bipopulation, both phases are used by the two subpopulations [21]. While the balance between exploration and exploitation is promoted by this parallel model, there is no explicit control over either phase. In spite of previous success, explicitly controlling the balance between exploration and exploitation is difficult and challenging because we lack a comprehensive understanding of when and how to control exploration and exploitation during a search process. Thus, we aim to study these two issues to provide some analyses and convincing conclusions.

In this paper, we propose a new search framework based on a bipopulation model with explicit exploration and exploitation control. In this framework, the population is split into two subpopulations for exploration and exploitation. To achieve an appropriate balance between them in the whole population, individuals in each subpopulation are driven by either an exploration or exploitation operator. Individuals migrate between the two subpopulations according to three transference strategies: 1) exploitation activation, 2) exploration activation, and 3) exploitation enhancement. To examine the performance of the proposed framework, an exploration-oriented operator, i.e., differential evolution, and an exploitation-oriented operator, i.e., adaptive Gaussian local search

with reinitialization, are embedded into the proposed framework and tested on 29 CEC2017 benchmark functions. The experimental results show that the proposed framework can significantly improve the search behavior of the original operator and balance exploration and exploitation well in a whole search period.

The main contributions of this paper include that:

- A bipopulation consisting of an exploration subpopulation and an exploitation subpopulation is introduced. Both subpopulation sizes are integers that vary in the range of zero to the fixed whole population size. This dynamic structure allows adaptive intensity of exploration and exploitation when different problems are being solved.
- A triple-transference strategy is proposed to explicitly control exploration and exploitation by migrating individuals between the two subpopulations during a search process. This strategy increases the frequency of interaction between explorers and exploiters, thus promoting the desired balance between these two phases.
- A new search framework based on the proposed bipopulation structure and the triple-transference strategy is established. Furthermore, a new algorithm named TRADE, which embeds differential evolution and Gaussian local search into this framework, is proposed. The experimental results of TRADE on 29 frequently used benchmark functions show that TRADE is highly competitive against and even better than most multipopulation or adaptation-based differential evolution methods.
- Some notable and valuable findings of this paper can be summarized as follows: 1) the balance between exploration and exploitation can be better controlled between two different types of search operators than that with a single one; and 2) a search framework performing exploration and exploitation in parallel is better than a serial one.

The remainder of this paper is organized as follows: Section 2 describes the related work on exploration and exploitation control in differential evolution. Section 3 presents the proposed triple-transference-based differential evolution method. Section 4 shows the experimental results. The discussion and conclusion are presented in Sections 5 and 6, respectively.

2. Related work

In this section, we provide a brief introduction to the mechanisms of canonical differential evolution (DE), including mutation, crossover, and selection. Additionally, we present some approaches that aim to enhance the search performance of DE by regulating the balance between exploration and exploitation.

2.1. The canonical differential evolution

DE has attracted extensive attention due to its simplicity and excellent performance in continuous numerical optimization problems [22]. DE begins with a randomly initialized population of N individuals, represented by $X_1, X_2, \ldots, X_N$ (N is the *population size*), and then goes into a loop, which consists of mutation, crossover, and selection, to improve the current population iteratively. In mutation, a *mutant* vector $V_i^{(t)}$ corresponding to an individual or *target* vector $X_i^{(t)}$ at the tth iteration is created by a specific strategy. The five most frequently used mutation strategies are as follows:

$$\text{DE/rand/1}: V_i^{(t)} = X_{r_1}^{(t)} + F \cdot (X_{r_2}^{(t)} - X_{r_3}^{(t)}). \tag{1}$$

$$\text{DE/rand/2}: V_i^{(t)} = X_{r_1}^{(t)} + F \cdot (X_{r_2}^{(t)} - X_{r_3}^{(t)}) + F \cdot (X_{r_4}^{(t)} - X_{r_5}^{(t)}). \tag{2}$$

$$\text{DE/best/1}: V_i^{(t)} = X_{best}^{(t)} + F \cdot (X_{r_1}^{(t)} - X_{r_2}^{(t)}). \tag{3}$$

$$\text{DE/best/2}: V_i^{(t)} = X_{best}^{(t)} + F \cdot (X_{r_1}^{(t)} - X_{r_2}^{(t)}) + F \cdot (X_{r_3}^{(t)} - X_{r_4}^{(t)}). \tag{4}$$

$$\text{DE/current-to-best/1}: V_i^{(t)} = X_i^{(t)} + F \cdot (X_{best}^{(t)} - X_i^{(t)}) + F \cdot (X_{r_1}^{(t)} - X_{r_2}^{(t)}). \tag{5}$$

$X_{r_1}^{(t)}$, $X_{r_2}^{(t)}$, $X_{r_3}^{(t)}$, $X_{r_4}^{(t)}$, and $X_{r_5}^{(t)}$ are mutually exclusive individuals randomly chosen from the current population, and all are different from $X_i^{(t)}$. $X_{best}^{(t)}$ represents the best individual (which has the lowest fitness in a minimization problem) at the tth iteration. The parameter F, called the $scale\ factor$, is a random value generated in the range (0, 1).

After mutation, each *mutant* vector mixes with its corresponding *target* vector to form an offspring or *trial* vector $U_i^{(t)}$ through crossover. The most commonly used crossover method in recent years is *binomial*. In it, a parameter named the *crossover rate* (C_r) controls each decision variable in the *mutant* vector to exchange with its *target* vector. This scheme can be shown as follows:

$$u_{i,j}^{(t)} = \begin{cases} v_{i,j}^{(t)} & \text{if } j = j^* \text{ or } r \le C_r, \\ x_{i,j}^{(t)} & \text{otherwise,} \end{cases} \tag{6}$$

where $v_{i,j}^{(t)}$, $x_{i,j}^{(t)}$, and $u_{i,j}^{(t)}$ are the jth dimensions of $V_i^{(t)}$, $X_i^{(t)}$, and $U_i^{(t)}$, respectively. j^*, an integer randomly chosen from the range $[1, D]$, ensures that $U_i^{(t)}$ obtains at least one variable from $V_i^{(t)}$. D represents the number of decision variables or $dimension$ of the problem. r is a random value generated in the range $[0, 1]$.

Finally, selection determines whether the offspring and parent survive into the next iteration in terms of $fitness$. The selection method is described as follows:

$$X_i^{(t+1)} = \begin{cases} U_i^{(t)} & \text{if } f\left(U_i^{(t)}\right) \le f\left(X_i^{(t)}\right), \\ X_i^{(t)} & \text{otherwise}, \end{cases} \tag{7}$$

where $f(\cdot)$ represents the objective function to be minimized.

2.2. Exploration and exploitation in DE

As it is a type of evolutionary algorithm, achieving a proper balance between exploration and exploitation also affects the search performance of DE. This balance is mainly influenced by mutation/crossover and the corresponding parameters (N, F, and C_r). Different mutation strategies have different characteristics of exploration or exploitation. For example, 'DE/rand/1' uses more exploration than 'DE/current-to-best/1' and 'DE/best/1'. To achieve a good balance between exploration and exploitation, much work has been done to introduce or integrate other mutation strategies. For example, JADE introduces a new mutation strategy named 'DE/current-to-pbest/1' [23]. Unlike 'DE/current-to-best/1' in canonical DE, each individual in JADE is guided by the top-p best individuals to disperse the attraction of the best individual. This strategy can prevent the whole population from quickly falling into the same local optimum. In addition, an extra archive mechanism helps JADE make use of historical experience and self-adaptive control parameters. EFADE proposes a triangular mutation operator to balance exploration and exploitation [24]. It employs a convex combination vector as the basic vector and utilizes three difference vectors between the tournament best, better, and worst of the chosen vectors to generate mutant vectors. To ensure a proper balance between exploration and exploitation, EDE and EBDE introduce two mutation strategies named 'ord_best' and 'ord_pbest', respectively [25]. Both approaches arrange the three vectors x_{best} (or x_{pbest}), x_{r_1}, and x_{r_2} based on their fitness values. Compared with using one mutation strategy, multistrategy-based DE can deal with complex problems better. For example, EPSDE maintains a mutation pool and a parameter pool in the search process [26]. EPSDE selects an appropriate mutation strategy and the corresponding parameters from the mutation pool and parameter pool, respectively, in each iteration. Similarly, CoDE adopts a mutation pool and parameter pool [27]. Unlike EPSDE, in which only one trial vector is created for each target vector, CoDE makes each individual create three trial vectors by using three different mutation strategies and a set of parameters randomly selected from the pool. AMECoDEs uses a composite mutation strategy guided by multiple elites to maintain a balance between exploration and exploitation [28]. This strategy utilizes 'DE/current-to-nbest' and 'DE/current-to-pbest' to generate two mutant vectors for each individual. The better mutant vector is then chosen as the final trial vector after undergoing binomial crossover.

The population size N of early DE is often specified by the user and kept fixed throughout the search process [29]. However, the setting of N highly depends on the problem to be solved and its dimension. A relatively high population size is preferred to deal with large local optima in hybrid composition functions and real-world problems. In recent years, a number of DE variants with flexible population sizes have been proposed [30]. Compared to N, determining the proper settings of F and C_r has a greater direct impact on the search performance of DE. Much work has also been done to self-adaptively control these two parameters. For example, SHADE stores historical F and C_r values that have succeeded in previous search rounds, and it generates new F and C_r values by sampling from the distribution of the stored values [31].

Population diversity and topology have also been studied for DE. In [32], an automatic population enhancement scheme was proposed to aid DE in escaping from local minima and stagnation. This strategy reinitializes the dimension with a standard deviation lower than a given threshold to improve diversity. Some structures, including cellular, hierarchical, and neighborhood structures, have been embedded into DE to improve exploration or exploitation [33]. In addition to these fixed population structures that do not change from beginning to end, adaptive structures have greater opportunity to balance exploration and exploitation. For example, NaDE provides a pool of index-based neighborhood topologies, and the neighborhood relationship of individuals is adaptively selected during the search process [34]. This method can ensure an appropriate neighborhood topology for each individual to adaptively match different phases of the search process. Recently, some distributed topology-based DE methods have shown good performance in balancing exploration and exploitation. For example, the population of MPEDE is divided into three small subpopulations and one large subpopulation [35]. The three subpopulations use three different mutation strategies to create offspring, and the strategy with the best performance after a specific measurement will be adopted in the large subpopulation. EDEV uses the same population topology as MPEDE but adopts JADE, CoDE, and EPSDE instead of the three mutation strategies in MPEDE [36].

3. Triple-Transference-Based Differential Evolution (TRADE)

In this section, we first explain why the proposed TRADE method adopts a parallel approach, which enables the simultaneous execution of exploration and exploitation. Then, we describe the overall framework of TRADE in detail by defining three transference strategies and explaining how they control exploration and exploitation in TRADE. Finally, we provide a detailed description of the exploration and exploitation operators in TRADE.

3.1. Motivation

In many EAs, the balance between exploration and exploitation is controlled by a single process, which results in two uncoordinated phases [4]. These algorithms are often designed to start with exploration and then turn to exploitation, gradually or suddenly, under the control of a unique process. Therefore, their exploration and exploitation are population-level processes that are mutually

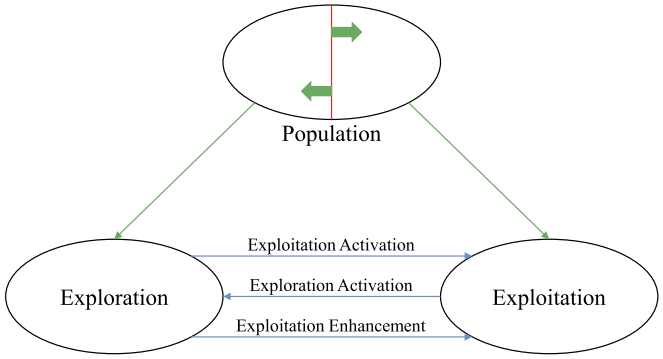


Fig. 1. Bi-population and triple-transference-based search framework.

Algorithm 1: TRADE.

Function TRADE(N, λ, η, T):
  Randomly initialize N individuals in a population P;
  Evaluate N individuals in the population;
  Randomly divide the population into P_{ei} with $N_{ei} = \lambda \times N$ and P_{er} with $N_{er} = N - \lambda \times N$ individuals;
  Initialize the number of offspring n_s of each individual in P_{ei} to 1;
  Initialize the state S of each individual in P_{er} to False;
  Initialize σ of each individuals in P_{er} to 1;
  while $\tau < T$ **do**
    for $i = 1$ **to** N_{ei} **do**
      P_{ei}^i = MODE(P_{ei}^i, n_s^i);
    end
    for $i = 1$ **to** N_{er} **do**
      $(P_{er}^i, S_i, \sigma_i)$ = AGLS-r(P_{er}^i, $f(P_{er}^i)$, S_i, σ_i);
    end
    Find the best individual P^* in $P_{er} \cup P_{ei}$;
    Update n_s of each individuals in P_{ei} according to EiE and ErA strategies;
    Adjust P_{er} and P_{ei} according to EiA strategy;
  end
  return P^*;
end

exclusive. Although some multipopulation-based EAs perform exploration and exploitation with different operators, interaction between the two phases is rare, making them uncoordinated. In addition to pure exploration and exploitation, an interleaved phase where exploration and exploitation are performed in parallel might work better than a situation where the phases are separated. Following this hypothesis, we attempt to develop a new search framework in which explorers and exploiters search simultaneously and interact frequently.

3.2. Overall implementation of Triple-Transference-Based Differential Evolution

In the proposed framework, the population is split into two subpopulations P_{er} and P_{ei}. They are driven by two different search operators to perform exploration and exploitation in parallel, as shown in Fig. 1. The dominant behavior of the whole population depends on P_{er} and P_{ei}. Suppose that N_{er} and N_{ei} are the subpopulation sizes of P_{er} and P_{ei}, respectively. The exploration-led phase occurs when $N_{er} >>$ (is significantly larger than) N_{ei}, and the exploitation-led phase occurs when $N_{er} << N_{ei}$. A balance phase occurs when $N_{er} \approx N_{ei}$. A good ratio between N_{er} and N_{ei} should be ensured to achieve an appropriate balance between exploration and exploitation. Therefore, individuals in P_{er} and P_{ei} are transferred frequently according to explicit exploration and exploitation control for sharing the information obtained during the search process. This explicit control consists of three transference strategies: exploitation activation, exploitation enhancement, and exploration activation. Exploitation activation transfers individuals from P_{er} to P_{ei} to provide potential solutions that can be further exploited. Exploitation enhancement transfers individuals from P_{er} to P_{ei} to enhance exploitation around current promising solutions. Exploration activation transfers individuals from P_{ei} to P_{er} to escape from local optima and explore other regions. In the exploitation population, multioffspring-based differential evolution (MODE) is proposed to exploit promising regions by using the information from previously visited regions where high-quality solutions have been found. In the exploration population, an adaptive Gaussian local search method with reinitialization (AGLS-r) is adopted to explore new areas that have not been visited or have been rarely visited in previous searches. Algorithm 1 gives the pseudocode of the proposed TRADE.

3.3. Explicit exploration and exploitation control

Intuitively, better solutions are more likely to be found near the high-quality solutions than the low-quality ones. Some EAs that adopt local search for the global best value follow this hypothesis [16]. However, some studies (e.g., those concerning the opposite learning strategy [37]) reveal that better solutions can also be found near the solutions that do not have high quality and are far from the best one. Thus, exploitation should be considered around both solutions with high quality and those with quality that is not bad and that are distant from high-quality solutions. The **Exploitation Activation (EiA)** strategy aims to drive P_{ei} to fine-tune around the potential solutions found by P_{er}. Specifically, EiA($u*$) represents transferring an individual $u*$ from P_{er} to P_{ei}, i.e.,

$$u^* \in \begin{cases} P_{ei}, & r < \varphi \\ P_{er}, & \text{Otherwise} \end{cases} \tag{8}$$

where u^* denotes the best individual (that with the minimum objective value) of P_{er} thus far. Once transferred to P_{ei}, u^* will perform exploitation in the next iteration. r represents a random number following a uniform distribution over $[0, 1]$. φ is a threshold that controls the probability of accepting a solution. A large φ transfers many individuals from P_{er} to P_{ei}, causing inadequate exploration, while a small φ cannot ensure effective exploitation. In this strategy, $\varphi = \frac{N_{er}}{N}$ is used to control the trade-off, where N_{er} is the size of the exploration population and N is the size of the whole population.

Recent studies show that the search performance can be improved by reusing the experience of previous searches. Some methods use the information (e.g., parameter values) of parent solutions that fail in a selection process to maintain population diversity [23]. Some consider the information of success history to adapt parameters and achieve good performance [31]. Following these successful experiments, the **Exploitation Enhancement (EiE)** strategy aims to reuse the valuable information of an exemplar in a promising region. Specifically, EiE(u, P_{ei}^{v*}) represents transferring the worst individual u^- from P_{er} to P_{ei} by increasing the number of offspring that belong to an improved individual $v*$ in the next iteration; i.e.,

$$u^- \in \begin{cases} P_{ei}, & I \neq \varnothing \wedge N_{er} > \theta \\ P_{er}, & \text{Otherwise} \end{cases} \tag{9}$$

where I is a set that consists of all successful individuals in P_{ei}. θ is a parameter that controls the frequency of EiE. If θ is small, P_{ei} may increase quickly, leading to stagnation. Otherwise, it may slow convergence. Thus, a time-varying θ is adopted to control the trade-off, i.e., $\theta = N \times e^{1-T/(T+1-\tau)}$, where N is the total population size, T is the maximum number of function evaluations, and τ is the current number of function evaluations. Note that the proposed strategy does not increase the space complexity in comparison with those using an archive among the aforementioned history-based algorithms.

Premature convergence has been recognized as one of the well-known drawbacks of EAs [38]. It occurs when the population is in locations in the search space that are outside the global optimal region. Although EiA provides diverse individuals for exploitation, EiE may drive the population to a few local optimal regions, which causes premature convergence. To avoid this problem, the number of iterations with no improvement, as simple but useful information, has been utilized to detect premature convergence [15]. Similarly, an **Exploration Activation (ErA)** strategy is proposed to delay convergence. Specifically, ErA(v) represents transferring an individual from P_{ei} to P_{er}, i.e.,

$$v \in \begin{cases} P_{er}, & h(v) > \eta \\ P_{ei}, & \text{Otherwise} \end{cases} \tag{10}$$

where η is a threshold that controls the living time of an individual; the value of η is discussed in section 5.4. $h(\cdot)$ is a function that counts the number of iterations in which v has ceased improving. The initial value of $h(v)$ is 0, and it will be increased by 1 if there is no improvement in v. Note that $h(v)$ is reset to 0 when there is an improvement in v. It should be noted that all offspring of an individual that satisfies this condition are also transferred to R_{er}. The only exception is that if the best individual in P_{ei} satisfies this condition, ErA will be applied to one of its offspring, and the number of its offspring will decrease by 1. This mechanism can ensure that the best individual is successively transferred for a sufficiently long time to guide exploitation.

To help understand the proposed triple-transference strategy, an example is given in Fig. 2, where (a) gives an initial state with $N_{er} = N_{ei} = 5$ and (b), (c) and (d) show the final state after performing EiA(u_2), EiE(u_5, v_5) and ErA(v_2), respectively. In Fig. 2(b), a distant but potential individual u_2 is transferred from P_{er} to P_{ei} and will share its information with other individuals in P_{ei} in the subsequent search. Exploitation activation can provide new information to guide the search of P_{ei} and can alleviate slow premature convergence. In Fig. 2(c), the number of offspring for v_5, which may be a promising solution, is increased. The more offspring an individual has, the more sufficient the search around its neighborhood. Exploitation enhancement can speed up convergence and increase the information diffusion of promising individuals. In Fig. 2(d), an individual v_2 that has failed to improve for a long time is transferred to P_{er} to reduce the attraction of the individual, which can be considered to be trapped in a local optimal region, and release its attraction to other individuals. Exploration activation can address the situation of oscillating in multiple local optimal regions.

3.4. Exploitation operator: Multioffspring-Based Differential Evolution (MODE)

Exploitation aims to find better solutions around promising regions where good solutions have been found. DE can maintain high-frequency information interchange in the population and result in powerful exploitation. In most DE variants, one parent usually breeds one offspring. This method first ensures a fixed population size in each generation and then makes each offspring randomly

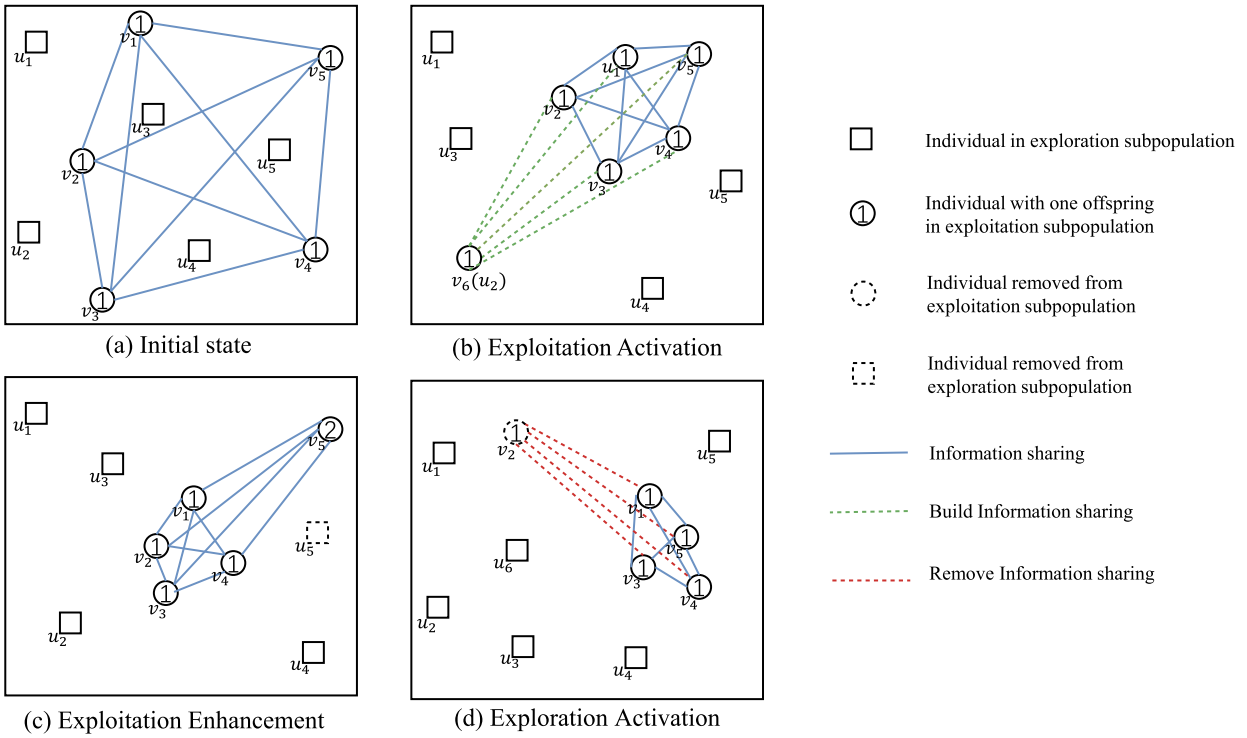


Fig. 2. Example of three explicit exploration and exploitation control.

Algorithm 2: MODE.

```
Function MODE(X, n_s):
    O = ∅;
    for i = 1 to n_s do
        r1, r2, r3: randomly select from P_er;
        V = r_1 + F · (r_2 − r_3);
        U = Crossover(V, X);
        O = O ∪ U;
    end
    Evaluate n_s offspring in O and find the best one O*;
    X = selection(X, O*);
    return X;
end
```

inherit some information from its own parent by crossover in DE. However, this one-offspring-based generation method limits the inheritance of valuable information because the crossover between the parent (target vector) and mutant vector will be performed only once and at random in each generation in DE.

To improve the exploitation in the neighborhood of promising individuals, a multioffspring-based generation method is proposed. In this method, one individual in DE can generate more than one offspring, and the number of offspring of an individual depends on its search ability. Specifically, the number of offspring for an individual that can find better results will be increased according to EiE. Otherwise, it will be decreased according to ErA. The proposed multioffspring method can enable DE to further exploit a promising individual. Notably, although a similar idea was proposed in CoDE [27], that one parent can breed three offspring by three different mutation strategies, our method is quite different from it in two respects. First, the number of offspring for an individual is determined by two transference strategies rather than a fixed value as in CoDE. Second, all offspring of an individual are generated by the DE/rand/1 strategy rather than multiple strategies (this will be discussed in Section 5.6). The pseudocode of the proposed multioffspring-based differential evolution (MODE) method is given in Algorithm 2.

3.5. Exploration operator: adaptive Gaussian local search with reinitialization

In DE, offspring and parent compete to survive into the next iteration according to the selection operator. Although this method can preserve poor solutions to maintain population diversity, some promising regions are also abandoned because of poor fitness. This situation is called failed exploration in [39]. To address failed exploration, adaptive Gaussian local search with reinitialization

(AGLS-r) is proposed. Local search techniques are often used to fine-tune neighborhoods of promising solutions in EAs for continuous optimization [16]. This characteristic can increase the living time of an individual whose current fitness is not good but that may be located in a promising region. At the individual level, local search can exploit the neighborhood of an individual. However, at the population level, local search for many distant individuals can improve exploration and address the situation of failed exploration. The pseudocode of AGLS-r is given in Algorithm 3. In it, there are two types of individuals. In the first type, individuals that belong to P_{er} in the last iteration have their states S marked as $false$. In the second type, individuals are transferred from P_{ei} to P_{er} according to ErA, and their states S are marked as $true$. Each individual with a false S uses Gaussian local search with mean 0 and variance 1, formulated as:

$$X' = X + \sigma \cdot N(0,1) \tag{11}$$

where σ represents the step size that controls the search scope of Gaussian local search. Otherwise, the position of the individual is reinitialized in the search space, and its state S is set to $false$. For an individual, if it can find a better solution after performing Gaussian local search, σ will be increased in the next search, aiming to enlarge the search region. Otherwise, σ will be decreased. $\hat{\sigma}$ and $\check{\sigma}$ are the upper bound and lower bound that constrain σ. In TRADE, $\hat{\sigma} = 20$ and $\check{\sigma} = 10^{-8}$ are adopted.

Algorithm 3: AGLS-r.

```
Function AGLS-r(X, f(X), S, σ):
    if S then
        X' = L + (U − L) · U(0,1);
    else
        X' = X + σ · N(0,1);
    end
    if S then
        S = Flase;
        X = X';
        f(X) = f(X');
    else
        if f(X') ≤ f(X) then
            X = X';
            f(X) = f(X');
            σ = 1.1 · σ;
            if σ > σ̂ then
                σ = 1;
            end
        else
            σ = 0.9 · σ;
            if σ < σ̌ then
                σ = 1;
            end
        end
    end
    return X, S, σ;
end
```

4. Experiment and results

The objective of this section is to provide evidence demonstrating that the proposed TRADE algorithm can achieve better exploration and exploitation when dealing with complex optimization problems. First, a detailed description of the experimental setup is provided, which includes the benchmark functions, comparison methods, and evaluation indices.

4.1. Experimental setup

To evaluate the performance of the proposed TRADE algorithm on different types of landscapes, 29 benchmark functions from the IEEE CEC2017 test suite were utilized. These functions consist of two unimodal functions (F1-F2), seven multimodal functions (F3-F9), ten hybrid functions (F10-F19), and ten composition functions (F20-F29). The search range for each function is $[-100, 100]^D$, where D is the dimension of the function. The maximum number of function evaluations (NFEs) is set to $10000 \times D$. The optimal results for each problem are defined as 100, 300, 400, ..., 2900, 3000, as stated in the related paper [40].

To analyze the results obtained from the experiments, several algorithms, including six state-of-the-art DE variants—LSHADE [41], MPEDE [35], EDEV [36], IMODE [42], CJADE-M [16], and APGSK_IMODE [43]—as well as the six winners of the CEC 2017 competition, were employed as comparative algorithms. Specifically, EBOwithCMAR uses a covariance matrix to enhance the local search ability of the original effective butterfly optimizer [44]. jSO is an improved variant of LSHADE that utilizes a new weighted version of a mutation strategy [45]. LSHADE-SPACMA hybridizes LSHADE-SPA with a modified version of CMA-ES [46]. HSES combines the covariance matrix adaptation-evolution strategy and a univariate sampling method [47]. LSHADE_RSP

Table 1
Parameter setting of six state-of-the-art DE variants.

Algorithm	Year	Parameters
LSHADE [41]	2014	$N = 18D, N^{min} = 4, H = 6, r^{arc} = 2.6, M_F = 0.5, M_{C_r} = 0.5, p = 0.11$
MPEDE [35]	2016	$N = 250,\ \lambda_1 = 0.2,\ ng = 20$
EDEV [36]	2018	$N = 100,\ \lambda_1 = \lambda_2 = \lambda_3 = 0.1,\ \lambda_4 = 0.7,\ ng = 20$
IMODE [42]	2020	$N = 6D^2,\ N_{min} = 4,\ A_r = 2.6,\ H = 20D$
CJADE-M [16]	2021	$N = 100,\ \mu_F = 0.5,\ \mu_{C_r} = 0.5,\ p = 0.05,\ c = 0.1,\ \rho = 0.998,\ L = 50$
APGSK_IMODE [43]	2021	$N = 30D, N^{min} = 4, H = 15D, r^{arc} = 1.4, M_F = 0.5, M_{C_r} = 0.5, p = 0.25$
TRADE	-	$N = 100,\ \lambda = 0.5,\ \eta = 1.5,\ F = 0.5,\ C_r = 0.9$

Table 2
Statistical results obtained via the Wilcoxon ran-sum test between TRADE and it peers on 29 benchmark functions of CEC2017.

	$D=10$		$D=30$		$D=50$		$D=100$	
	F1-F9	F10-F29	F1-F9	F10-F29	F1-F9	F10-F29	F1-F9	F10-F29
LSHADE	0/4/5	4/8/9	0/3/6	6/3/12	0/2/7	9/0/12	1/0/8	6/2/13
MPEDE	4/4/1	15/5/0	4/1/4	12/1/7	5/0/4	15/0/5	5/0/4	9/1/10
EDEV	0/7/2	14/3/3	3/2/4	14/1/5	4/0/5	12/1/7	4/0/5	9/2/9
IMODE	2/5/2	12/3/5	6/0/3	17/0/3	5/0/4	17/0/3	5/0/4	10/0/10
CJADE-M	0/4/5	9/8/3	3/2/4	15/3/2	4/0/5	16/1/3	4/0/5	10/1/9
APGSK_IMODE	1/5/3	8/5/8	8/0/1	19/0/1	6/1/2	19/0/1	6/1/2	15/2/3
EBOwithCMAR	0/4/5	5/5/10	0/2/7	6/1/13	1/0/8	7/2/11	0/0/9	3/3/14
jSO	0/4/5	3/7/10	1/3/5	4/2/14	1/0/8	5/3/12	0/0/9	3/1/16
LSHADE_SPACMA	0/4/5	6/6/8	0/2/7	6/1/13	0/0/9	7/1/12	0/0/9	3/3/14
HSES	0/4/5	12/4/4	0/2/7	5/5/10	1/0/8	7/1/12	0/0/9	3/2/15
LSHADE_RSP	0/4/5	3/8/9	1/3/5	5/1/14	0/0/9	4/3/13	0/0/9	3/0/17
ELSHADE_SPACMA	0/4/5	4/7/9	0/3/6	7/3/10	0/0/9	6/4/10	0/0/9	5/1/14

Table 3
Statistical results obtained via the Friedman test between TRADE and it peers on 29 benchmark functions of CEC2017.

	$D=10$	$D=30$	$D=50$	$D=100$
TRADE	6.9561	7.1968	7.3955	8.8654
LSHADE	5.4124	5.8999	5.9222	6.5510
MPEDE	10.4929	7.9128	9.0142	9.1474
EDEV	8.9588	8.2089	7.9479	8.5757
IMODE	8.7776	11.3905	11.3739	10.7049
CJADE-M	7.9632	9.2975	9.5838	9.1254
APGSK_IMODE	7.1139	11.2705	11.6491	11.9953
EBOwithCMAR	**5.0216**	4.7684	4.9811	5.3215
jSO	5.4530	4.5700	4.7715	5.4368
LSHADE_SPACMA	6.1964	5.1711	4.8729	3.7647
HSES	7.7721	5.3861	**4.2471**	**2.7863**
LSHADE_RSP	5.6295	5.5632	4.9797	4.2295
ELSHADE_SPACMA	5.2525	**4.3644**	4.2610	4.4959

improves LSHADE by implementing a rank-based selective pressure strategy [48]. ELSHADE_SPACMA improves LSHADE_SPACMA by dynamically adjusting the p value in the mutation strategy [49]. The statistical results are summarized in Table 2 and Table 3, and the mean and standard deviation are presented in Tables A1-A4 in the Appendix.

Table 1 lists the parameter settings of the six state-of-the-art DE variants, which follow the recommended configurations in the literature. Each algorithm runs 51 times independently on each function, and statistical results are obtained. The results are the mean and standard deviation of the errors between the optimal solution found by an algorithm and the known global optimum. Statistical analyses are conducted with the Wilcoxon rank-sum test at a significance level of $\alpha = 0.05$ [50]. All experiments are implemented in MATLAB software on a PC with a 3.00 GHz Intel(R) Core(TM) i5-7400 CPU and 8 GB RAM. The related code and data can be found at https://toyamaailab.github.io/.

4.2. Results of numerical optimization

The statistical results obtained via the Wilcoxon rank-sum test and the Friedman test between TRADE and its peers on the 29 functions of CEC2017 are given in Table 2 and Table 3, respectively. The symbol + indicates that the proposed algorithm is better than the comparative algorithm, while − means the opposite. The symbol ≈ indicates that there is no significant difference between them. Thus, the value of +/≈ /− gives the number of times the proposed method wins/ties/loses in comparison with its peers on the

Table 4
Number of the best value obtained by TRADE and six winners on 29 benchmark functions of CEC2017.

	$D=10$		$D=30$		$D=50$		$D=100$	
	F1-F9	F10-F29	F1-F9	F10-F29	F1-F9	F10-F29	F1-F9	F10-F29
TRADE	5	11	3	3	2	4	0	6
EBOwithCMAR	8	8	6	8	6	1	3	1
jSO	7	11	4	3	4	2	2	0
LSHADE_SPACMA	6	10	8	4	4	1	5	0
HSES	8	4	6	7	8	10	7	11
LSHADE_RSP	7	7	4	6	4	3	3	1
ELSHADE_SPACMA	5	12	4	4	4	1	4	1

29 benchmark functions. The complete results, including the mean and standard deviation of the errors, are summarized in Tables A1-A4 in the Appendix, and the best value obtained is in bold. The results of $+/\approx/-$ in Table 2 on each dimension are grouped into problems F1-F9 and F10-F29. The former are relatively simple problems, including three unimodal and seven multimodal functions, while the latter are complex problems, including ten hybrid and ten composition functions.

From Table 2, we can observe that TRADE performs worse than LSHADE, EDEV, CJADE-M, and APGSK_IMODE, as well as all winners on the simple problems in 10 dimensions. However, TRADE outperforms MPEDE, EDEV, IMODE, CJADE-M, and HSES on 75%, 70%, 60%, 45%, and 60% of the complex problems in 10 dimensions, respectively. In 30 dimensions, TRADE is competitive with MPEDE, EDEV, and CJADE-M and performs better than APGSK_IMODE on 90% of the simple problems; on 60%, 70%, 85%, 75%, and 95% of the complex problems, it outperforms MPEDE, EDEV, IMODE, CJADE-M, and APGSK_IMODE, respectively. Similarly, in 50 dimensions, TRADE outperforms MPEDE, EDEV, IMODE, CJADE-M, and APGSK_IMODE on 75%, 60%, 85%, 80%, and 95% of the complex functions, respectively. Although TRADE's performance is reduced in 100 dimensions, it is still competitive with MPEDE, EDEV, IMODE, CJADE-M, and APGSK_IMODE on both simple and complex problems. These results demonstrate TRADE's excellent search performance, particularly on hybrid and composition functions that require a well-balanced combination of exploration and exploitation. While TRADE does not outperform the six winners in terms of total problems, it significantly surpasses EBOwithCMAR, jSO, LSHADE_SPACMA, HSES, LSHADE_RSP, and ELSHADE_SPACMA on eight, six, seven, eight, four, and six functions in 50 dimensions, respectively. This indicates that TRADE is the best method for a subset of the problems. Compared to these winners, TRADE only uses fixed parameter settings of F and C_r; thus, its performance can be further enhanced by using parameter adaptation or other mutation strategies in the future. From Table 3, we can observe that TRADE ranks 7th, 8th, 8th, and 9th out of 13 algorithms in 10, 30, 50, and 100 dimensions, respectively. Specifically, TRADE outperforms MPEDE, EDEV, IMODE, CJADE-M and APGSK_IMODE.

To further assess the search capability of the proposed TRADE method, Table 4 shows the number of best results obtained by each algorithm; detailed results can be found in Tables A5-A8 in the Appendix. From Table 4, we can see that in the case of 10 dimensions, TRADE is capable of finding 11 best results out of 20 complex functions, which is more than is achieved by EBOwithCMAR, LSHADE_SPACMA, HSES, and LSHADE_RSP. For 50 dimensions, TRADE is second only to one algorithm in finding the greatest number of best results. It is noteworthy that in the case of 100 dimensions, TRADE can find 6 best results for the 20 complex functions, while EBOwithCMAR, LSHADE_RSP, and ELSHADE_SPACMA can find only one best result, and jSO and LSHADE_SPACMA cannot find any best results. These findings demonstrate the remarkable search ability of TRADE on complex functions that require a balance between exploration and exploitation.

To further analyze the performance of the proposed TRADE, the graph of convergence and the box-and-whisker plot of 13 DE variants on four different types of functions from CEC2017 with 30 dimensions are given in Figs. 3 and 4, respectively. More graphs with 10, 50, and 100 dimensions are given in Figs. S1-S6 in the Appendix. Specifically, a unimodal function (F1), a simple multimodal function (F4), two hybrid functions (F15 and F16), and two composition functions (F26 and F28) are adopted to perform the analysis. Fig. 3 shows that the convergence rate of TRADE is similar to that of other DE variants in the early stages of the search process. However, compared to other DE variants, the convergence rate of TRADE increases in the middle stages of the search process, as can be clearly observed in Fig. 3(b), (c), (e), and (f). This phenomenon indicates that TRADE has the ability to jump out of local optima due to the proposed explicit exploration and exploitation control. Additionally, when facing complex problems, TRADE is less likely to fall into local optima than other DE variants. In Fig. 4, it is obvious that the solutions provided by TRADE are not only of high quality but also stable.

5. Discussion

In this section, we discuss several aspects of the proposed TRADE method, including computational complexity analysis, individual transference and population diversity analysis, exploration and exploitation analysis, parameter analysis, and component analysis. These discussions provide evidence showing why TRADE performs well on complex optimization problems.

5.1. Computational complexity analysis

In this section, the corresponding time complexity of TRADE is analyzed as follows: First, the initialization of N individuals costs $O(N \times D)$. Then, the overall time complexity of performing exploration and exploitation is $O(N \times D)$ because the total population size is not changed during the search process. Then, the evaluation of these individuals in the whole population is $O(N)$. The time

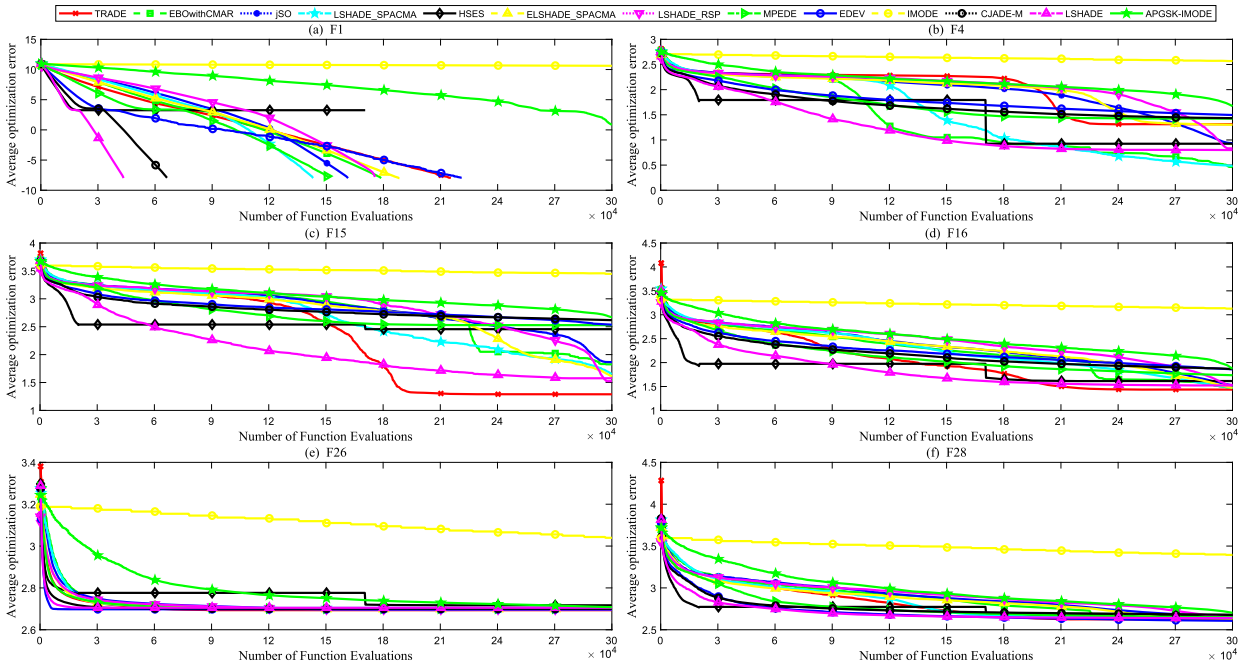


Fig. 3. Convergence figure of average optimization errors of DE variants on six functions of CEC2017 with 30 dimensions.

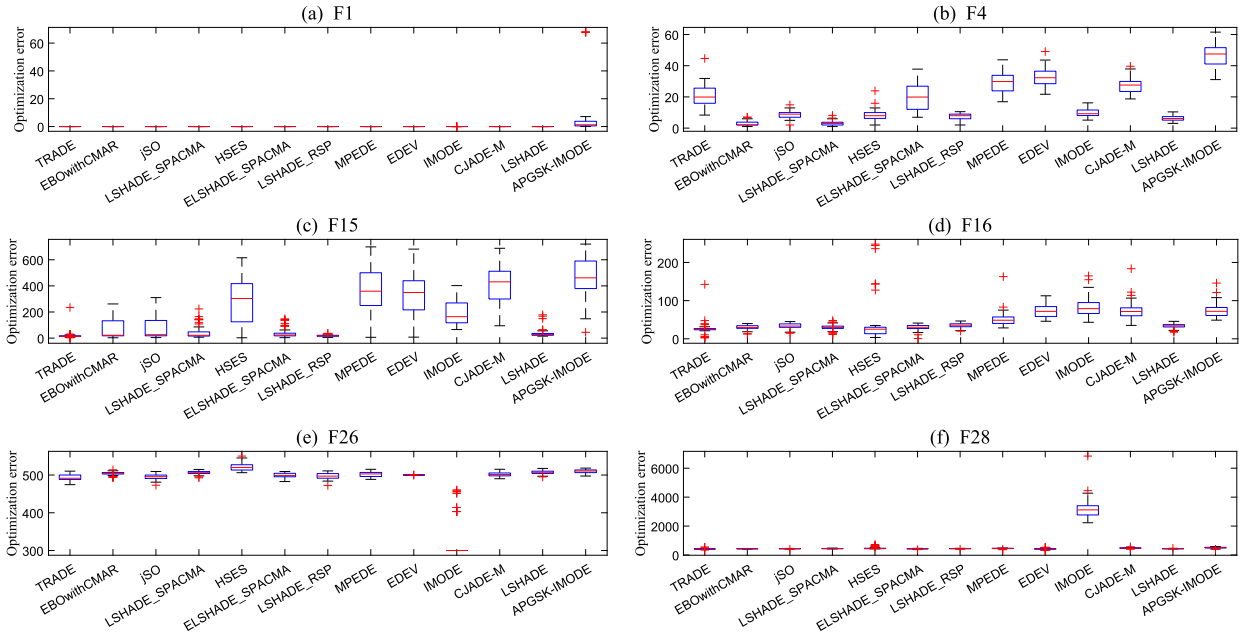


Fig. 4. Box-and-whisker diagrams of optimization errors of DE variants on six functions of CEC2017 with 30 dimensions.

complexities of EiA, EiE, and ErA are $O(N)$, $O(N)$, and $O(2N)$, respectively. As a result, the overall time complexity of the proposed TRADE is $O(N \times D)$.

5.2. Individual transference and population diversity in TRADE

To further analyze the proposed strategy, individual transference and population diversity are considered as the two main indices. Individual transference can reveal the main process of the search, and it is expressed by the number of individuals in P_{er} and P_{ei} in each iteration. Population diversity describes the degree of dispersion of the population in each iteration, and it can be calculated as:

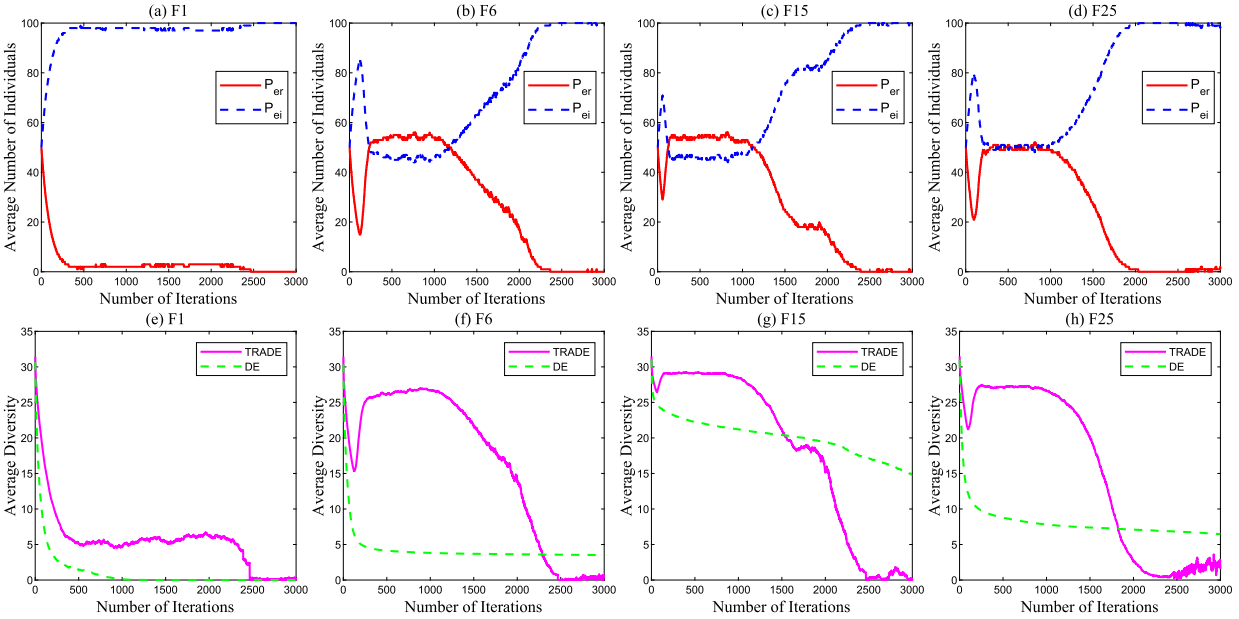


Fig. 5. Individual transference and population diversity on four types of functions.

$$\tilde{D} = \frac{1}{N}\sqrt{\sum_{i=1}^{N}\left(\left\|X_i - \frac{1}{N}\sum_{j=1}^{N} X_j\right\|_2\right)^2} \tag{12}$$

Fig. 5 shows the number of individuals and the population diversity of four typical examples on unimodal, multimodal, hybrid, and composition functions. These two indices are the average values of the 51 independent experiments. From Fig. 5(a), we can see that individuals are transferred from P_{er} to R_{ei} in the first 500 iterations, and almost all individuals belong to P_{ei} at the 2500th iteration. This reveals that TRADE focuses on exploitation after performing fast exploration, which is consistent with our usual observation that algorithms generally need not explore the search space of simple unimodal functions that have no local optima. Additionally, from Fig. 5(e), we find that the average population diversity of TRADE is higher than that of DE. The main reason is that it can search in many distinct areas simultaneously, but DE can only search in a few areas that are close to each other. Fig. 5(b) and (f) give the individual transference and population diversity for F6, which is characterized by a large number of local optima. To find the global region among a huge number of local regions, exploration should be performed more intensively than on unimodal functions. Furthermore, algorithms are required to jump out of local optima once individuals are trapped. Similar observations are made in Fig. 5(c) and (d) for F15, which hybridizes four basic functions. Compared with unimodal and multimodal functions, hybrid functions are more difficult to solve because of the characteristics of different properties around different local optima. In fact, due to the complex properties of hybrid functions, exploration should be adequately performed. Specifically, in Fig. 5(d), the great change around the 100^{ih} iteration should be noted. This kind of change appears when exploitation fails to find better solutions in areas where the current search is performed. Therefore, ErA transfers individuals to exploration to search new potential areas. This behavior reveals that the proposed strategy can help DE jump out of local optima. From Fig. 5(g), we can see that, compared with DE, TRADE not only maintains better population diversity in the early and middle stages of the search but also rapidly decreases population diversity for powerful exploitation at the end of the search. Finally, it is also illuminating to examine the individual transference and population diversity for composition functions, i.e., F25. The composition functions are the most complex set in the CEC2017 test suite because their complex properties include being multimodal, nonseparable, and asymmetrical and having different properties around different local optima. To address this kind of function, algorithms are expected to have the ability to perform full exploration, escape from local optima, and conduct powerful exploitation. Fig. 5(d) and (h) explain why TRADE can perform better than its peers on most of these composition functions. In Fig. 5(a), the search process of TRADE is divided into three stages. The first stage is from the beginning to approximately the 200th iteration. In this stage, TRADE focuses on exploring the new areas where better solutions can be found, and then tentative exploitation is performed in every potential area. The second stage is the balance stage, from approximately the 250th iteration to approximately the 1200th iteration. In this stage, although individuals are exchanged frequently, N_{er} and N_{ei} are almost equal. The last stage is after the 1200th iteration, during which TRADE focuses on exploiting the most promising areas found in the previous stage.

5.3. Exploration and exploitation in TRADE

To conduct a comprehensive evaluation of the trade-off between exploration and exploitation in TRADE, we utilize a diversity metric for population-based metaheuristic algorithms that is taken from [4]. In general, individuals with high quality tend to attract

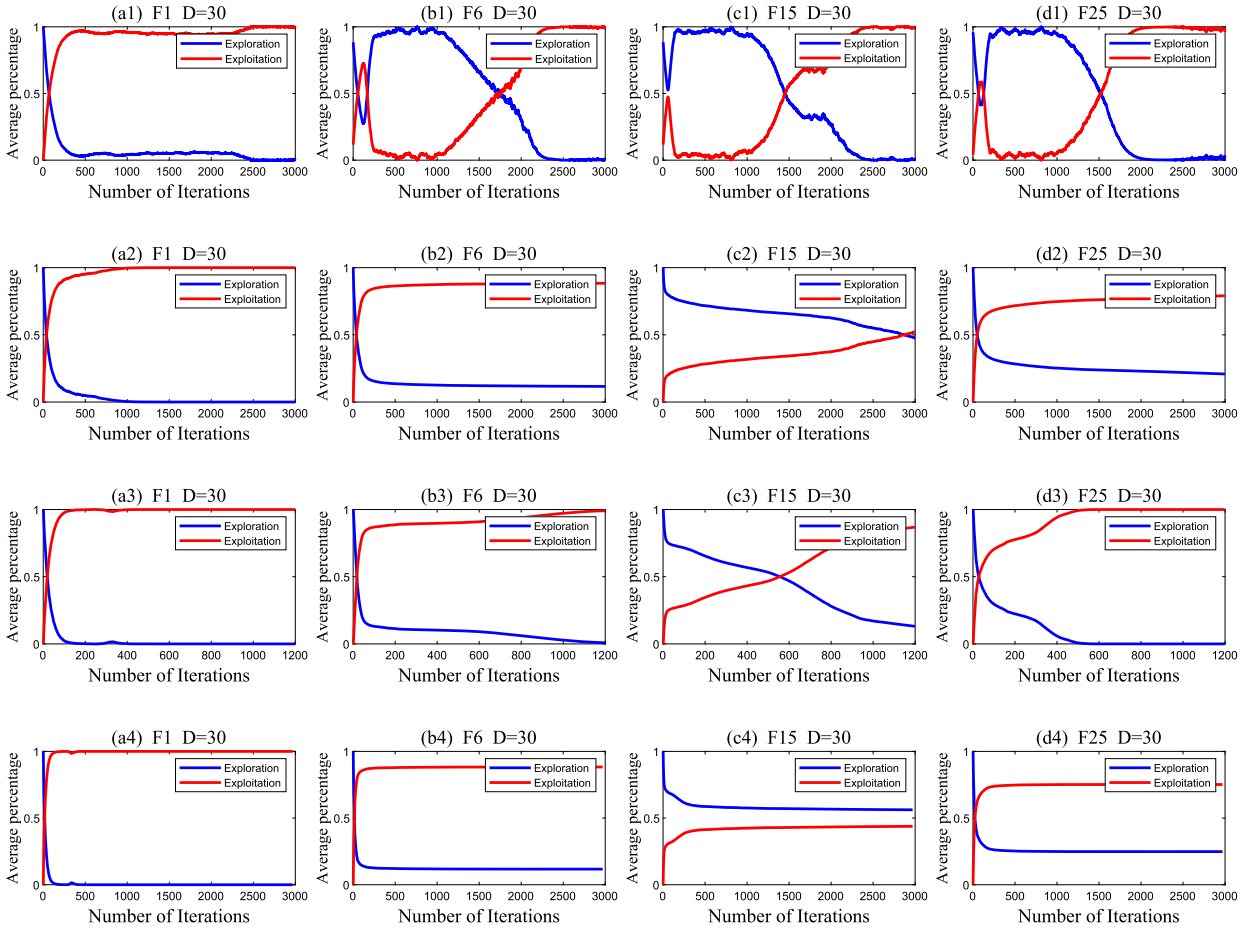


Fig. 6. The exploration and exploitation were evaluated on four types of functions, with (a1), (b1), (c1), and (d1) corresponding to TRADE, (a2), (b2), (c2), and (d2) corresponding to DE. (a3), (b3), (c3), and (d3) corresponding to MPEDE, and (a4), (b4), (c4), and (d4) corresponding to CJADE-M.

the search process toward them, resulting in a decrease in the distance between individuals and an increase in the effect of exploitation. Conversely, when the distance between individuals increases, the effect of the exploration process becomes more evident. This balance can be quantified as the percentages of exploration and exploitation, formulated as:

$$XPL\% = \left(\frac{Div}{Div_{max}}\right) \times 100 \tag{13}$$

$$XPT\% = \left(\frac{|Div - Div_{max}|}{Div_{max}}\right) \times 100 \tag{14}$$

where $XPL\%$ represents the level of exploration and $XPT\%$ corresponds to the level of exploitation. Div is the dimensionwise diversity, which can be calculated by Eqs. (15) and (16). Div_{max} represents the maximum diversity value in the entire optimization process.

$$Div = \frac{1}{m}\sum_{j=i}^{m} Div_j \tag{15}$$

$$Div_j = \frac{1}{n}\sum_{i=i}^{n} |x_i^j - median(x^j)| \tag{16}$$

where x_i^j is the jth dimension of the ith individual. n and m are the population size and dimension size, respectively. $median(x^j)$ represents the median value of the jth dimension in the whole population.

Fig. 6 illustrates the exploration and exploitation effects of TRADE, DE, MPEDE, and CJADE-M on four different types of functions (F1, F6, F15, and F25). As shown in Fig. 6(a1) and (a2), TRADE and DE exhibit the same average percentage of exploration and exploitation on F1. However, as seen in (b1) and (b2), DE tends to quickly shift from exploration to exploitation, while TRADE emphasizes exploration more in the first half of the search and exploitation more in the last half, likely because DE is prone to getting stuck in local optima, while TRADE can enable individuals to escape such optima through the ErA strategy. Moreover, (c1)

Table 5
Analysis of η in TRADE.

η	$0.1D$	$0.5D$	$1.0D$	$1.5D$	$2.0D$	$2.5D$
f_{best}	0	2	5	11	7	**12**
r_g	5.9483	4.5172	3.1379	**2.2931**	2.5172	2.5862
p-value	0	0.000006	0.085512	-	0.648239	0.550786
< 0.05	YES	YES	NO	-	NO	NO

Table 6
Analysis of λ in TRADE.

λ	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
f_{best}	5	5	4	4	7	**8**	6	4	4	6	6
r_g	6.9655	5.8966	6.3276	6.3103	5.6552	**5.1724**	5.7931	5.7586	6.8621	5.8621	5.3966
p-value	0.039523	0.405749	0.0184748	0.191389	0.579397	-	0.476077	0.500924	0.052388	0.428473	0.796918
< 0.05	YES	NO	YES	NO	NO	-	NO	NO	NO	NO	NO

and (c2) indicate that DE maintains a high level of exploration, while TRADE can focus on promising regions and benefit from the EiA strategy. Finally, while DE shows balanced exploration and exploitation in (d1) and (d2), it cannot obtain better results, whereas TRADE not only balances exploration and exploitation but also improves exploitation to achieve better results, as per the EiE strategy. Similarly, MPEDE adopts a multipopulation strategy to improve search ability, while CJADE-M improves exploitation by incorporating local search. However, the exploration and exploitation of MPEDE and CJADE-M are similar to those of DE, lacking exploration in the early stages of the search and lacking the ability to improve exploitation in the later stages. In contrast, TRADE maintains a well-balanced combination of exploration and exploitation, which is why it outperforms MPEDE and CJADE-M on complex optimization problems.

5.4. Parameter analysis

In this section, two important parameters, η and λ, that significantly affect the performance of TRADE are analyzed. η controls the living time of an individual that does not improve. It significantly affects the frequency with which an individual transfers from exploitation to exploration. In fact, this frequency depends on the dimension of the problem that needs to be solved. For example, problems with low dimensions need more exploration to explore the entire search space, while problems with high dimensions need more exploitation to find a better result. λ is another important parameter that determines the initial size of the exploitation population. The higher λ is, the more similar TRADE is to canonical DE in the early stage of the search. If λ is 0, TRADE begins with pure exploration. If λ is 1.0, it means pure exploitation is used in the beginning. Twenty-nine benchmark functions of CEC2017 with 30 dimensions are adopted to perform the analysis. For η, five values $\{0.1D, 0.5D, 1.0D, 1.5D, 2.0D, 2.5D\}$ are considered, where D represents the dimension of the problem. For λ, eleven values taken evenly from $[0, 1]$ are considered. Except for η and λ, all parameters are set the same as for TRADE in Table 1. Each configuration is run 25 times to obtain the statistical results. Tables 5 and 6 list some indices, including f_b (the number of functions that achieve the best result), $p-value$ (via the Wilcoxon signed-rank test), and r_g (the general ranking). Figs. 7 and 8 show the graph of individual migration for each configuration on F25 of CEC2017 with 30 dimensions.

In Table 5, we can see that TRADE with a large η performs better than that with a low η, according to the values of f_{best} and r_g. TRADE performs the best when $\eta = 1.5D$, and it is significantly better than the others when $\eta = 0.1D$, $0.5D$, and $1.0D$. Figs. 7(a) and (b) give the 3-D map and contour map of F25 in the 2-D search space. From Fig. 7(c), we can find excessive exploration and insufficient exploitation because a low η enables individuals to frequently jump out of local regions. Compared with low η, a relatively large η, such as $1.5D$ or $2.0D$, can ensure appropriate exploration in the early state of a search and sufficient exploitation around promising regions. In summary, $\eta = 1.5D$ is recommended.

In Table 6, we can see that TRADE with $\lambda = 0.5$ finds the best solution on the eight functions and ranks first according to f_{best} and r_g. However, there are no significant differences between $\lambda = 0.5$ and most of the other values according to the p-value and the graph of individual transfers from Fig. 8. According to the overall performance, $\lambda = 0.5$ is suggested.

5.5. Exploitation enhancement analysis of TRADE

The exploitation enhancement (EiE) strategy significantly affects the exploitation of TRADE. In this section, we analyze three variants of TRADE, including TRADE/noEiE, TRADE/p, and TRADE/o. In TRADE/noEiE, EiE is removed, which means that each individual in P_{ei} can create only one offspring, as in canonical DE. In TRADE/p, EiE transfers individuals from P_{er} to P_{ei} by inserting a copy of a randomly selected improved parent individual instead of increasing the number of offspring for it. Similarly, TRADE/o inserts a copy of the offspring of a randomly selected improved parent individual. These three variants can help us to analyze the effectiveness of EiE and the proposed multioffspring-based generation strategy. Each variant runs 25 times on 29 benchmark functions from CEC2017 with 30 dimensions. Table 7 gives the statistical result via the Wilcoxon rank sum test at the significance value $\alpha = 0.05$.

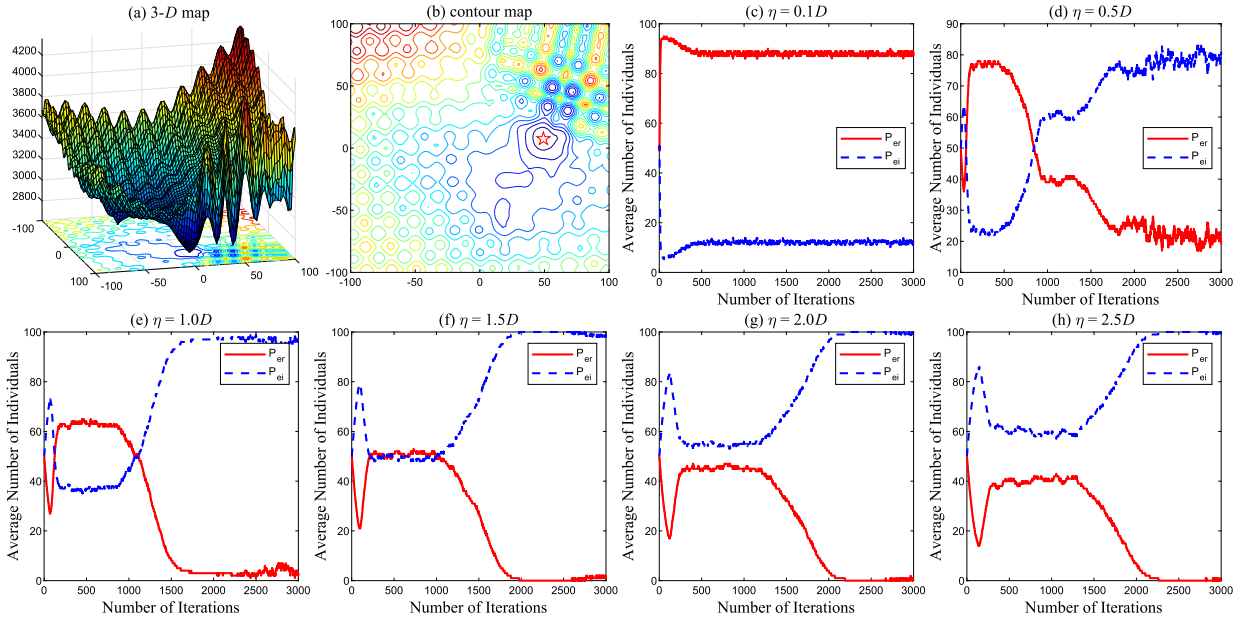


Fig. 7. Individual transference of TRADE with different settings of η on F25 of CEC2017 with 30 dimensions.

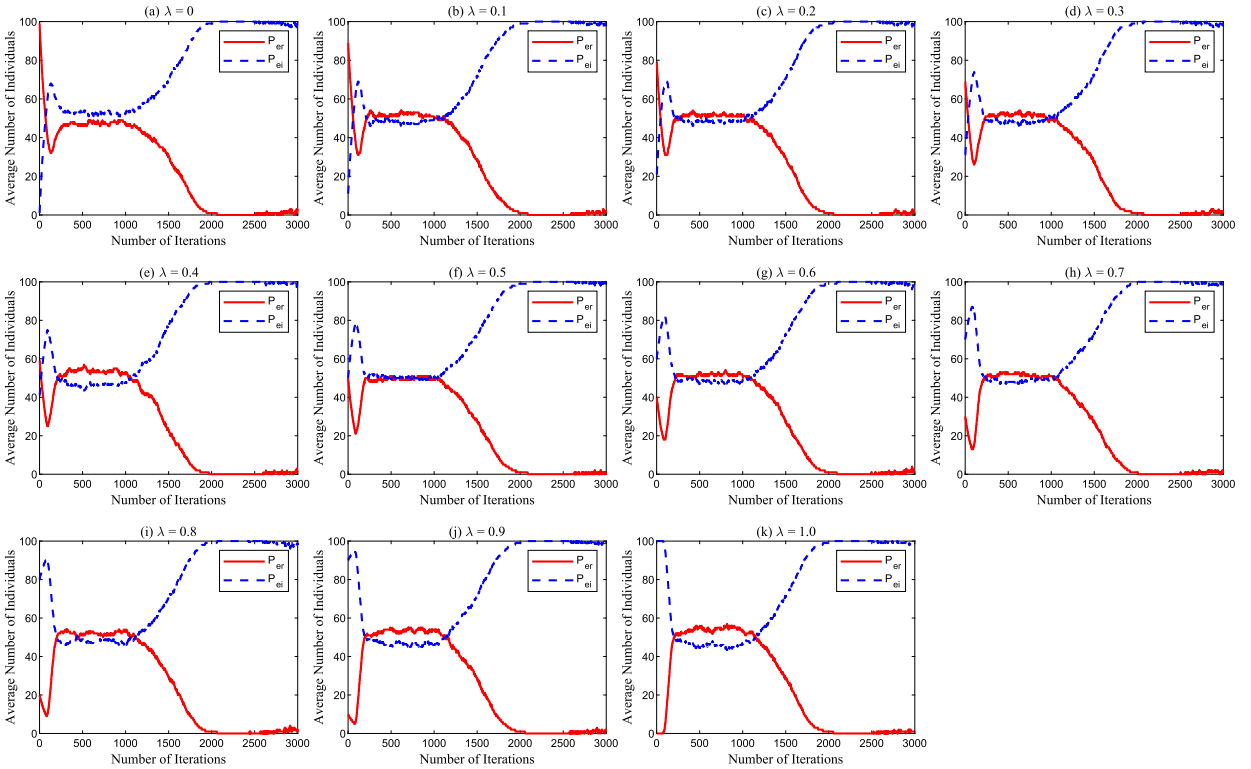


Fig. 8. Individual transference of TRADE with different settings of η on F25 of CEC2017 with 30 dimensions.

Table 7
Statistical result via the Wilcoxon rank sum test between TRADE and three variants.

	TRADE/noEiE	TRADE/p	TRADE/o
+/≈ /−	14/15/0	19/10/0	15/11/3

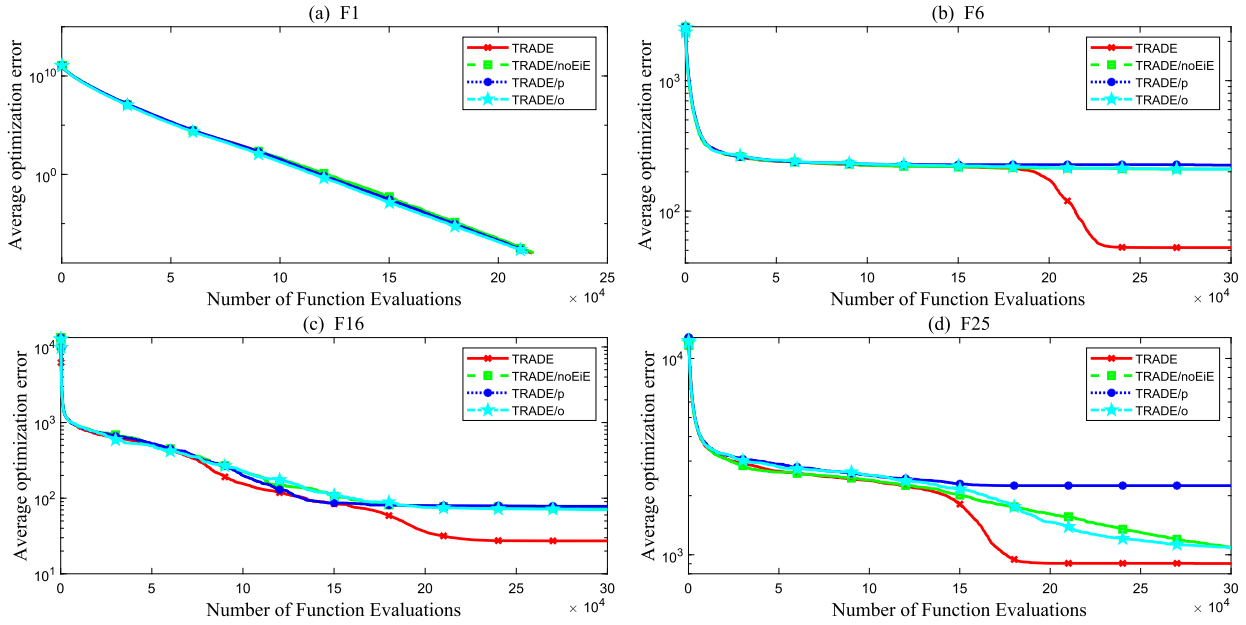


Fig. 9. Convergence figure of average optimization errors of TRADE variants on four functions of CEC2017 with 30 dimensions.

Table 8
Statistical result via the Wilcoxon rank sum test between TRADE and four variants.

	TRADE/best/1	TRADE/current-to-pbest/1	TRADE/current-to-rand/1	TRADE/rand-to-current/1
+/≈/−	27/0/2	27/1/1	25/3/1	25/3/1

Table 7 shows that TRADE with EiE is significantly better than that without EiE and than that without the multioffspring-based generation strategy. From the graph of convergence shown in Fig. 9, we can see that TRADE has a more powerful exploitation ability than its variants.

5.6. Mutation analysis of TRADE

In the proposed TRADE, DE/rand/1 is adopted to perform exploitation. In this section, we analyze four other variants, TRADE/best/1, TRADE/current-to-pbest/1, TRADE/current-to-rand/1, and TRADE/rand-to-current/1, which embed four different mutation strategies into the proposed search framework. Table 8 shows the statistical results obtained by comparing TRADE/rand/1 with its four variants.

From Table 8, we can see that DE/rand/1 performs significantly better than the other mutation strategies in the proposed search framework. Fig. 10 shows that the other mutation strategies quickly converge and have difficulty escaping from local optima. The reason is that these mutation strategies have specific directions, such as to *best* or *pbest*. Having a direction improves exploitation, but the exploitation enhancement strategy also intensifies exploitation and leads to premature convergence.

5.7. Population size of TRADE

In the proposed TRADE algorithm, a fixed population size is utilized. However, the experimental results from numerical optimization indicate that TRADE performs worse than some algorithms that use a reduced population size strategy [41]. In this section, we investigate whether this strategy is effective for improving TRADE's performance. The strategy employed in TRADE is similar to other methods, with the only difference being that TRADE utilizes two subpopulations. When the population size needs to be reduced, we first identify the worst individual from each of the two subpopulations and then remove the worst individual from the pair. The maximum and minimum population sizes are set to $18D$ and 4, respectively, which are consistent with the settings in other algorithms, where D represents the dimension of the problem. The experiment is conducted across all dimensions of the 29 benchmark functions in the IEEE CEC2017 test suite. Table 9 presents the statistical results obtained via the Wilcoxon rank-sum test, comparing TRADE with LTRADE (TRADE with the population reduction strategy).

According to the results presented in Table 9, it is evident that TRADE outperforms LTRADE across all dimensions. This suggests that the population reduction strategy is ineffective for improving TRADE's performance. One possible explanation for this is that the reduced population size disrupts the balance between exploration and exploitation that is maintained by the proposed transference

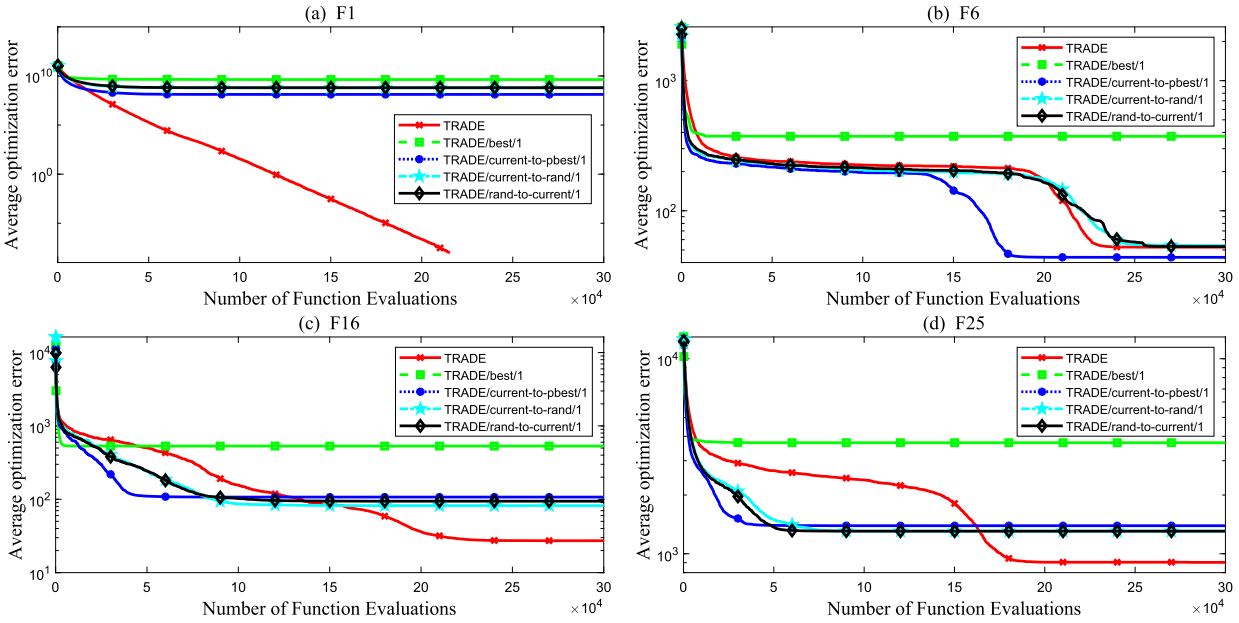


Fig. 10. Convergence figure of average optimization errors of TRADE variants on four functions of CEC2017 with 30 dimensions.

Table 9
Statistical result via the Wilcoxon rank sum test between TRADE and LTRADE.

	$D=10$	$D=30$	$D=50$	$D=100$
LTRADE	6/21/2	18/7/4	22/3/4	20/4/5

strategies. Specifically, the population reduction strategy may result in the removal of potentially valuable individuals from one subpopulation who are ready to be transferred into the other subpopulation.

6. Conclusion

In this section, we summarize our contributions and present some issues that should be studied in our future work. This paper presents a new search framework based on a bipopulation model and a triple-transference strategy for the EA community. The proposed framework employs two different types of search operators, MODE and AGLS-r, to cooperatively search decision variable space. The three individual transference strategies can be used to effectively balance the two operators according to the information provided from the previous search. The experimental results of TRADE show that the framework can significantly improve the search performance of DE, even better than most of the state-of-the-art DE variants, such as MPEDE, EDEV, IMODE, CJADE-M and APGSK_IMODE.

Our future work involves three areas: 1) In addition to fitness, other information, such as diversity and entropy, should be utilized to determine the exploration and exploitation levels. 2) Many individuals that are far away from each other are generated by our exploration-to-exploitation transference strategy, and their topology should be further studied for more effective information exchange to determine directions with more potential in the next search. 3) It would be interesting to extend the proposed framework to solve multiobjective evolutionary optimization problems [51].

CRediT authorship contribution statement

Zonghui Cai: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Xiao Yang:** Software, Writing – review & editing. **MengChu Zhou:** Conceptualization, Software, Writing – review & editing. **Zhi-Hui Zhan:** Conceptualization, Software, Writing – review & editing. **Shangce Gao:** Conceptualization, Methodology, Software, Supervision, Writing – review & editing.

Declaration of competing interest

The authors certify that they have NO affiliations with or involvement in any organization or entity with any financial interest (such as honoraria; educational grants; participation in speakers' bureaus; membership, employment, consultancies, stock ownership,

or other equity interest; and expert testimony or patent-licensing arrangements), or non-financial interest (such as personal or professional relationships, affiliations, knowledge or beliefs) in the subject matter or materials discussed in this manuscript.

Data availability

Data will be made available on request.

Acknowledgement

This research was partially supported by the Japan Society for the Promotion of Science (JSPS) KAKENHI under Grant JP22H03643, Japan Science and Technology Agency (JST) Support for Pioneering Research Initiated by the Next Generation (SPRING) under Grant JPMJSP2145, and JST through the Establishment of University Fellowships towards the Creation of Science Technology Innovation under Grant JPMJFS2115.

Appendix A. Supplementary material

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ins.2023.119656.

References

[1] S. Gao, M. Zhou, Y. Wang, J. Cheng, H. Yachi, J. Wang, Dendritic neural model with effective learning algorithms for classification, approximation, and prediction, IEEE Trans. Neural Netw. Learn. Syst. 30 (2019) 601–614.
[2] A.E. Eiben, C.A. Schippers, On evolutionary exploration and exploitation, Fundam. Inform. 35 (1998) 35–50.
[3] M. Črepinšek, S.-H. Liu, M. Mernik, Exploration and exploitation in evolutionary algorithms: a survey, ACM Comput. Surv. 45 (2013) 1–33.
[4] B. Morales-Castañeda, D. Zaldivar, E. Cuevas, F. Fausto, A. Rodríguez, A better balance in metaheuristic algorithms: does it exist?, Swarm Evol. Comput. (2020) 100671.
[5] G. Karafotias, M. Hoogendoorn, Á.E. Eiben, Parameter control in evolutionary algorithms: trends and challenges, IEEE Trans. Evol. Comput. 19 (2014) 167–187.
[6] S.-H. Liu, M. Mernik, B.R. Bryant, To explore or to exploit: an entropy-driven approach for evolutionary algorithms, Int. J. Knowl. Based Intell. Eng. Syst. 13 (2009) 185–206.
[7] V. Osuna-Enciso, E. Cuevas, B.M. Castañeda, A diversity metric for population-based metaheuristic algorithms, Inf. Sci. 586 (2022) 192–208.
[8] C. Segura, C.A.C. Coello, E. Segredo, A.H. Aguirre, A novel diversity-based replacement strategy for evolutionary algorithms, IEEE Trans. Cybern. 46 (2015) 3233–3246.
[9] C. Huang, X. Zhou, X. Ran, Y. Liu, W. Deng, W. Deng, Co-evolutionary competitive swarm optimizer with three-phase for large-scale complex optimization problem, Inf. Sci. 619 (2023) 2–18.
[10] M. Wineberg, J. Chen, The shifting balance genetic algorithm as more than just another island model GA, in: Genetic and Evolutionary Computation Conference, Springer, 2004, pp. 318–329.
[11] Y. Wang, Y. Yu, S. Gao, H. Pan, G. Yang, A hierarchical gravitational search algorithm with an effective gravitational constant, Swarm Evol. Comput. 46 (2019) 118–139.
[12] W.H. Lim, N.A.M. Isa, Particle swarm optimization with adaptive time-varying topology connectivity, Appl. Soft Comput. 24 (2014) 623–642.
[13] D.H. Wolpert, W.G. Macready, No free lunch theorems for optimization, IEEE Trans. Evol. Comput. 1 (1997) 67–82.
[14] W. Xiang, S. Ma, M. An, hABCDE: a hybrid evolutionary algorithm based on artificial bee colony algorithm and differential evolution, Appl. Math. Comput. 238 (2014) 370–386.
[15] Y.-J. Gong, J.-J. Li, Y. Zhou, Y. Li, H.S.-H. Chung, Y.-H. Shi, J. Zhang, Genetic learning particle swarm optimization, IEEE Trans. Cybern. 46 (2015) 2277–2290.
[16] S. Gao, Y. Yu, Y. Wang, J. Wang, J. Cheng, M. Zhou, Chaotic local search-based differential evolution algorithms for optimization, IEEE Trans. Syst. Man Cybern. Syst. 51 (2021) 3954–3967.
[17] Q.H. Nguyen, Y.-S. Ong, M.H. Lim, A probabilistic memetic framework, IEEE Trans. Evol. Comput. 13 (2009) 604–623.
[18] M. Črepinšek, M. Mernik, S.-H. Liu, Analysis of exploration and exploitation in evolutionary algorithms by ancestry trees, Int. J. Innov. Comput. Appl. 3 (2011) 11–19.
[19] S.-H. Liu, M. Mernik, D. Hrnčič, M. Črepinšek, A parameter control method of evolutionary algorithms using exploration and exploitation measures with a practical application for fitting Sovova's mass transfer model, Appl. Soft Comput. 13 (2013) 3792–3805.
[20] Z. Cai, S. Gao, X. Yang, G. Yang, S. Cheng, Y. Shi, Alternate search pattern-based brain storm optimization, Knowl.-Based Syst. 238 (2022) 107896.
[21] W. Gao, G.G. Yen, S. Liu, A cluster-based differential evolution with self-adaptive strategy for multimodal optimization, IEEE Trans. Cybern. 44 (2013) 1314–1327.
[22] M. Pant, H. Zaheer, L. Garcia-Hernandez, A. Abraham, et al., Differential evolution: a review of more than two decades of research, Eng. Appl. Artif. Intell. 90 (2020) 103479.
[23] J. Zhang, A.C. Sanderson, JADE: adaptive differential evolution with optional external archive, IEEE Trans. Evol. Comput. 13 (2009) 945–958.
[24] A.W. Mohamed, P.N. Suganthan, Real-parameter unconstrained optimization based on enhanced fitness-adaptive differential evolution algorithm with novel mutation, Soft Comput. 22 (2018) 3215–3235.
[25] A.W. Mohamed, A.A. Hadi, K.M. Jambi, Novel mutation strategy for enhancing SHADE and LSHADE algorithms for global numerical optimization, Swarm Evol. Comput. 50 (2019) 100455.
[26] R. Mallipeddi, P.N. Suganthan, Q.-K. Pan, M.F. Tasgetiren, Differential evolution algorithm with ensemble of parameters and mutation strategies, Appl. Soft Comput. 11 (2011) 1679–1696.
[27] Y. Wang, Z. Cai, Q. Zhang, Differential evolution with composite trial vector generation strategies and control parameters, IEEE Trans. Evol. Comput. 15 (2011) 55–66.
[28] L. Cui, G. Li, Z. Zhu, Q. Lin, K.-C. Wong, J. Chen, N. Lu, J. Lu, Adaptive multiple-elites-guided composite differential evolution algorithm with a shift mechanism, Inf. Sci. 422 (2018) 122–143.
[29] A.P. Piotrowski, Review of differential evolution population size, Swarm Evol. Comput. 32 (2017) 1–24.
[30] W. Zhu, Y. Tang, J.-a. Fang, W. Zhang, Adaptive population tuning scheme for differential evolution, Inf. Sci. 223 (2013) 164–191.
[31] R. Tanabe, A. Fukunaga, Success-history based parameter adaptation for differential evolution, in: 2013 IEEE Congress on Evolutionary Computation, IEEE, 2013, pp. 71–78.

[32] M. Yang, C. Li, Z. Cai, J. Guan, Differential evolution with auto-enhanced population diversity, IEEE Trans. Cybern. 45 (2014) 302–315.
[33] N. Lynn, M.Z. Ali, P.N. Suganthan, Population topologies for particle swarm optimization and differential evolution, Swarm Evol. Comput. 39 (2018) 24–35.
[34] Y. Cai, G. Sun, T. Wang, H. Tian, Y. Chen, J. Wang, Neighborhood-adaptive differential evolution for global numerical optimization, Appl. Soft Comput. 59 (2017) 659–706.
[35] G. Wu, R. Mallipeddi, P.N. Suganthan, R. Wang, H. Chen, Differential evolution with multi-population based ensemble of mutation strategies, Inf. Sci. 329 (2016) 329–345.
[36] G. Wu, X. Shen, H. Li, H. Chen, A. Lin, P.N. Suganthan, Ensemble of differential evolution variants, Inf. Sci. 423 (2018) 172–186.
[37] Y. Xu, Z. Yang, X. Li, H. Kang, X. Yang, Dynamic opposite learning enhanced teaching–learning-based optimization, Knowl.-Based Syst. 188 (2020) 104966.
[38] H.M. Pandey, A. Chaudhary, D. Mehrotra, A comparative review of approaches to prevent premature convergence in GA, Appl. Soft Comput. 24 (2014) 1047–1077.
[39] S. Chen, A. Bolufé-Röhler, J. Montgomery, T. Hendtlass, An analysis on the effect of selection on exploration in particle swarm optimization and differential evolution, in: 2019 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2019, pp. 3037–3044.
[40] N.H. Awad, M.Z. Ali, J.J. Liang, B.Y. Qu, P.N. Suganthan, Problem definitions and evaluation criteria for the CEC2017 special session and competition on single objective real-parameter numerical optimization, Technical Report, Nanyang Technological University, Singapore, 2017.
[41] R. Tanabe, A.S. Fukunaga, Improving the search performance of shade using linear population size reduction, in: 2014 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2014, pp. 1658–1665.
[42] K.M. Sallam, S.M. Elsayed, R.K. Chakrabortty, M.J. Ryan, Improved multi-operator differential evolution algorithm for solving unconstrained problems, in: 2020 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2020, pp. 1–8.
[43] A.W. Mohamed, A.A. Hadi, P. Agrawal, K.M. Sallam, A.K. Mohamed, Gaining-sharing knowledge based algorithm with adaptive parameters hybrid with imode algorithm for solving CEC 2021 benchmark problems, in: 2021 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2021, pp. 841–848.
[44] A. Kumar, R.K. Misra, D. Singh, Improving the local search capability of effective butterfly optimizer using covariance matrix adapted retreat phase, in: 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 1835–1842.
[45] J. Brest, M.S. Maučec, B. Bošković, Single objective real-parameter optimization: algorithm jSO, in: 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 1311–1318.
[46] A.W. Mohamed, A.A. Hadi, A.M. Fattouh, K.M. Jambi, LSHADE with semi-parameter adaptation hybrid with CMA-ES for solving cec 2017 benchmark problems, in: 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 145–152.
[47] G. Zhang, Y. Shi, Hybrid sampling evolution strategy for solving single objective bound constrained problems, in: 2018 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2018, pp. 1–7.
[48] V. Stanovov, S. Akhmedova, E. Semenkin, LSHADE algorithm with rank-based selective pressure strategy for solving CEC 2017 benchmark problems, in: 2018 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2018, pp. 1–8.
[49] A.A. Hadi, A.W. Mohamed, K.M. Jambi, Single-objective real-parameter optimization: enhanced LSHADE-SPACMA algorithm, in: Heuristics for Optimization and Learning, 2021, pp. 103–121.
[50] J. Carrasco, S. García, M. Rueda, S. Das, F. Herrera, Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: practical guidelines and a critical review, Swarm Evol. Comput. 54 (2020) 100665.
[51] S. Han, K. Zhu, M. Zhou, X. Liu, H. Liu, Y. Al-Turki, A. Abusorrah, A novel multiobjective fireworks algorithm and its applications to imbalanced distance minimization problems, IEEE/CAA J. Autom. Sin. 9 (2022) 1476–1489.